Assignment 8

CS 6117 (Fall 2025)

Exercise 1. Suppose that we have two monads S and T modeling two kinds of computational effects. A natural question is whether we can define a single monad combining both kinds of computational effects.

There are different ways of combining monads and effects; here is one approach. Given two monads (S, μ^S, η^S) and (T, μ^T, η^T) on a category $\mathbf{C}$, we can try to define natural transformations such that $(S \circ T, \mu^{S\circ T}, \eta^{S\circ T})$ is a monad.

This is not always possible for all monads S and T, but one sufficient condition is when there is a *distributive law* of T over S: a natural transformation $\lambda : T \circ S \Rightarrow S \circ T$ satisfying some conditions; see Wikipedia ("Distributive law between monads") or nLab. The data in a distributive law can be used to define natural transformations $\mu^{S\circ T}$ and $\eta^{S\circ T}$ making $S \circ T$ into a monad. (If you are curious, try to define these natural transformations.)

1. Let $\mathbf{C}$ be $\mathbf{Set}$. Let $(M, \cdot, e)$ be a monoid (for instance, $M = \Sigma^*$ could be the list monoid over alphabet Σ), and let $S : \mathbf{C}$ be a set. Suppose that $\downarrow : S \times M \to S$ is a *right-action*:

   $$s \downarrow e = s \qquad s \downarrow (m_1 \cdot m_2) = (s \downarrow m_1) \downarrow m_2$$

   The *writer monad* for M is defined by $WX \triangleq M \times X$, with multiplication $\mu^W(m, (m', x)) = (m \cdot m', x)$, and unit $\eta^W(x) = (e, x)$. The *reader monad* for S is defined by $RY \triangleq S \Rightarrow Y$, with multiplication $\mu^R(f) = \lambda s.\, f(s)(s)$, and unit $\eta^R(x) = \lambda s.\, x$ the constant function.

   Define a natural transformation $\lambda : W \circ R \Rightarrow R \circ W$.

2. Check that your distributive law satisfies the four required diagrams.

In particular, if M is the monoid S^* and the right-action $s \downarrow (s_1 \dots s_n) = s_n$ simply keeps the last state, then we recover the usual state monad. In other words, a computation that can read and write is a computation in the state monad.

---

1. The writer monad W and reader monad R are defined on $\mathbf{Set}$ as specified. For a set X, define the component $\lambda_X : W(RX) \to R(WX)$ by$\lambda_X(m, f) = \lambda s.\ (m, f(s \downarrow m))$ where $(m, f) \in M \times (S \to X)$. To confirm naturality,

consider a function $h : X \to Y$. Then, $\lambda_Y \circ W(Rh)(m, f) = \lambda_Y(m, h \circ f) = \lambda s.\ (m, h(f(s \downarrow m)))$ and $R(Wh) \circ \lambda_X(m, f) = \lambda s.\ (m, h(f(s \downarrow m)))$ The expressions establish naturality.

2. A distributive law $\lambda : W \circ R \Rightarrow R \circ W$ must satisfy the following four:
   1. $\lambda \circ \eta^W R = R\eta^W$
   2. $\lambda \circ W\eta^R = \eta^R W$
   3. $\lambda \circ \mu^W R = R\mu^W \circ \lambda W \circ W\lambda$
   4. $\lambda \circ W\mu^R = \mu^R W \circ R\lambda \circ \lambda R$

   Verify each condition:

   Condition 1: For $f \in RX$, $\lambda(\eta^W(RX)(f)) = \lambda(e, f) = \lambda s.\ (e, f(s \downarrow e)) = \lambda s.\ (e, f(s))$ since $s \downarrow e = s$. The right side is $R\eta^W(f) = \lambda s.\ \eta^W(f(s)) = \lambda s.\ (e, f(s))$. They match.

   Condition 2: For $(m, x) \in WX$, $\text{z}\lambda(W\eta^R(m, x)) = \lambda(m, \eta^R(x)) = \lambda s.\ (m, \eta^R(x)(s \downarrow m)) = \lambda s.\ (m, x)$ And the right side is $\eta^R(WX)(m, x) = \lambda s.\ (m, x)$. They match.

   Condition 3: For $(m_1, (m_2, f)) \in WWRX$, $\lambda(\mu^W(RX)(m_1, (m_2, f))) = \lambda(m_1 \cdot m_2, f) = \lambda s.\ (m_1 \cdot m_2, f(s \downarrow (m_1 \cdot m_2)))$ For the right side, first $W\lambda(m_1, (m_2, f)) = (m_1, \lambda s.\ (m_2, f(s \downarrow m_2)))$. Then, $\lambda W(m_1, \lambda s.\ (m_2, f(s \downarrow m_2))) = \lambda t.\ (m_1, (m_2, f((t \downarrow m_1) \downarrow m_2)))$ Finally, $R\mu^W(\lambda t.\ (m_1, (m_2, f((t \downarrow m_1) \downarrow m_2)))) = \lambda s.\ (m_1 \cdot m_2, f((s \downarrow m_1) \downarrow m_2))$. By associativity of the action, they match.

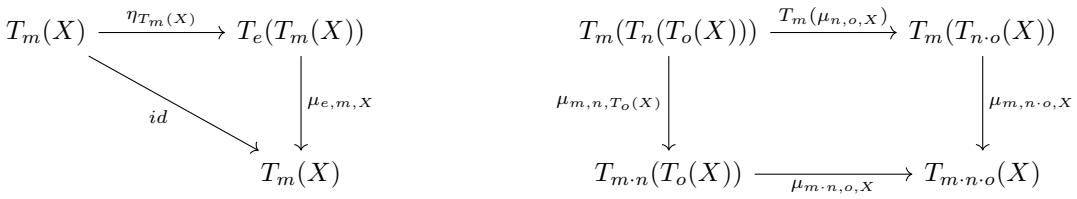
   Condition 4: For $(m, g) \in WRRX$, where $g : S \to (S \to X)$, $\lambda(W\mu^R(m, g)) = \lambda(m, \lambda s.\ g(s)(s)) = \lambda t.\ (m, g(t \downarrow m)(t \downarrow m))$. For the right side, $\lambda R(m, g) = \lambda s.\ (m, g(s \downarrow m))$. Then, $R\lambda(\lambda s.\ (m, g(s \downarrow m))) = \lambda u.\ \lambda v.\ (m, g(u \downarrow m)(v \downarrow m))$ Finally,$\mu^R WX(\lambda u.\ \lambda v.\ (m, g(u \downarrow m)(v \downarrow m))) = \lambda w.\ (m, g(w \downarrow m)(w \downarrow m))$. They match.

Exercise 2. On A7, you saw some examples of graded comonads. It can also be useful to consider *graded monads* following a similar idea, where there is an endofunctor $T_m : \mathbf{C} \to \mathbf{C}$ for every grade m in a pre-ordered monoid $(M, \cdot, e, \leq)$. There are several different (but equivalent) ways of defining M-graded monads; we consider a more concrete definition here (although the optional exercises below describe a more concise alternative).

A M-graded monad on $\mathbf{C}$ consists of the following data:

- For every grade $m \in M$, an endofunctor $T_m : \mathbf{C} \to \mathbf{C}$.
- A natural transformation $\eta : Id \Rightarrow T_e$.
- For every $m, n \in M$, a natural transformation $\mu_{m,n} : T_m \circ T_n \Rightarrow T_{m \cdot n}$.
- For every grades $m \leq n$, a natural transformation $c_{m \leq n} : T_m \Rightarrow T_n$.

Such that the following diagrams commute:



Intuitively, the grades can track quantitative information that can accumulate across multiple effectful computations; for instance, there are graded monads for tracking cost or resource usage.

In this exercise, we will work out a graded monad for tracking error accumulation and propagation. We will work in the category **PMet** of (extended, pseudo-) metric spaces, which is similar to the category **Met** we saw on A7 except that the reflexivity condition is relaxed to be:

- $d(x, x) = 0$

In particular, in a pseudometric space, there may be distinct points $x \neq y$ that are distance zero. Everything you showed in A7 for **Met** continues to work in this category **PMet**.

Our grading monoid will be $(\mathbb{R}^+, +, 0)$, the non-negative real numbers under addition. We will consider a graded monad such that on objects, we have:

$$T_r(X, d_X) \triangleq (\{(x, x') \mid d_X(x, x') \leq r\}, d_{T_rX})$$

where the distance is defined:

$$d_{T_rX}((x, x'), (y, y')) \triangleq d_X(x, y)$$

This is not a typo: the distance between two pairs is the distance between the *first* components only. The multiplication will be defined as:

$$\mu_{r,s,X}((x, x'), (y, y')) \triangleq (x, y')$$

Define the rest of the data to form a $\mathbb{R}$-graded monad on **PMet**. Note that you will need to check that all of the components of the natural transformations are morphisms in **PMet** (i.e., non-expansive), and you will also need to check that the multiplication $\mu_{r,s,X}$ indeed produces something in the target metric space $T_{r+s}X$.

Define the remaining components of the $\mathbb{R}^+$-graded monad on **PMet**. The unit natural transformation $\eta : \mathrm{Id} \Rightarrow T_0$ is defined by $\eta_X(x) = (x, x)$. Since $d_X(x, x) = 0 \leq 0$, $(x, x) \in T_0X$. It is non-expansive because $d_{T_0X}((x, x), (y, y)) = d_X(x, y) = d_X(x, y)$. For $r \leq s$, the natural transformation $c_{r\leq s} : T_r \Rightarrow T_s$ is defined by $c_{r\leq s,X}(x, x') = (x, x')$. If $(x, x') \in T_rX$, then $d_X(x, x') \leq r \leq s$, so $(x, x') \in T_sX$. It is non-expansive because $d_{T_sX}((x, x'), (y, y')) = d_X(x, y) =$

$d_{T_rX}((x,x'),(y,y'))$. The multiplication $\mu_{r,s,X} : T_r(T_sX) \to T_{r+s}X$, given by $\mu_{r,s,X}((x,x'),(y,y')) = (x,y')$, lands in $T_{r+s}X$ because $d_X(x,y') \leq d_X(x,y) + d_X(y,y') \leq r + s$, using the condition $d_X(x,y) \leq r$ from the outer pair and $d_X(y,y') \leq s$ from the inner pair. It is non-expansive because $d_{T_{r+s}X}((x,y'),(a,b')) = d_X(x,a) = d_{T_r(T_sX)}(((x,x'),(y,y')),((a,a'),(b,b')))$. The definitions align with the unit and associativity laws.

Exercise 3. We need two more pieces to adapt this graded monad to Fuzz.

1. In class (and in Moggi's paper), we saw that we need a *strong* monad in order to use a monadic type with contexts with more than one variable. A M-graded monad is (left) *strong* if there is a natural transformation $t_{m,A,B} : A \otimes T_mB \to T_m(A \otimes B)$ making the diagrams for a strong monad commute "modulo grading".

   Define the components $t_{m,A,B}$ for the graded error monad. You do not need to show naturality or that the (unspecified) diagrams commute, but again these should hold almost by definition.

2. The first exercise considered a distributive law of a monad over a monad. We will need a distributive law of a graded comonad D over a graded monad T. Let $D_r : \mathbf{PMet} \to \mathbf{PMet}$ be the scaling functor from A7. Show that there is a morphism:

   $$\lambda_{s,r,X} : D_s(T_r(X)) \to T_{s \cdot r}(D_s(X))$$

   Note that the grading on T changes. Of course, these maps should form the components of a natural transformation, and they should satisfy some diagrams that I won't even attempt to draw here, although you can look at Figure 8 in "Combining Effects and Coeffects via Grading" by Gaboardi, Katsumata, Orchard, Breuvart, and Uustalu (ESOP 2016) if curious.

3. Finally, use these maps along with any data from A7 to interpret the following typing rule:

   $$\frac{\Gamma \vdash e : T_p\sigma \qquad \Delta, x :_s \sigma \vdash f : T_q\tau}{s \cdot \Gamma + \Delta \vdash \text{let-bind } x = e \text{ in } f : T_{s \cdot p+q}\tau}$$

---

1. The strength is defined by $t_{m,(A,d_A),(B,d_B)}(a,(b,b')) = ((a,b),(a,b'))$, where $(a,(b,b')) \in A \otimes T_mB$. This is a morphism in **PMet** because $d_{T_m(A\otimes B)}(((a,b),(a,b')),((c,d),(c,d'))) = d_{A\otimes B}((a,b),(c,d)) = d_A(a,c) + d_B(b,d)$, and $d_{A\otimes T_mB}((a,(b,b')),(c,(d,d'))) = d_A(a,c) + d_B(b,d)$. The distances coincide, so it is non-expansive. The condition $(a,b'),(a,b') \in T_m(A \otimes B)$ holds since $d_{A\otimes B}((a,b),(a,b')) = d_B(b,b') \leq m$.

2. The scaling functor is $D_s(X, d_X) = (X, s \cdot d_X)$, where $(s \cdot d_X)(x, y) = s \cdot d_X(x, y)$. Then, $D_s(T_r X)$ has underlying set $\{(x, x') \mid d_X(x, x') \leq r\}$ and metric $s \cdot d_{T_r X}((x, x'), (y, y')) = s \cdot d_X(x, y)$. Similarly, $T_{s \cdot r}(D_s X)$ has underlying set $\{(x, x') \mid s \cdot d_X(x, x') \leq s \cdot r\} = \{(x, x') \mid d_X(x, x') \leq r\}$ and metric $d_{T_{s \cdot r}}((x, x'), (y, y')) = s \cdot d_X(x, y)$. The spaces coincide, so define $\lambda_{s,r,X}((x, x')) = (x, x')$. This is a morphism. Naturality follows as both functors commute with the identity on morphisms.

3. Assuming $[\![s \cdot \Gamma + \Delta]\!] = [\![\Delta]\!] \otimes D_s[\![\Gamma]\!]$, the semantics is $[\![\text{let-bind } x = e \text{ in } f]\!] = \mu_{s \cdot p, q, [\![\tau]\!]} \circ T_{s \cdot p}[\![f]\!] \circ t_{s \cdot p, [\![\Delta]\!], D_s[\![\sigma]\!]} \circ (\mathrm{id}_{[\![\Delta]\!]} \otimes \lambda_{s,p,[\![\sigma]\!]}) \circ (\mathrm{id}_{[\![\Delta]\!]} \otimes D_s[\![e]\!])$ where $[\![e]\!] : [\![\Gamma]\!] \to T_p[\![\sigma]\!]$ and $[\![f]\!] : [\![\Delta]\!] \otimes D_s[\![\sigma]\!] \to T_q[\![\tau]\!]$.

Exercise 4 (Category of the Week). Recall that every monad T on $\mathbf{C}$ is associated with two special categories: the *Kleisli* category $\mathbf{C}_T$, and the *Eilenberg-Moore (EM)* category $\mathbf{C}^T$.

The Kleisli category $\mathbf{C}_T$ has a more straightforward use in programming languages: a morphism from A to B in the Kleisli category is a morphism from A to TB in the base category $\mathbf{C}$, which we think of as a program that accepts data of type A and produces data of type B, while possibly performing some effects described by the monad T.

The EM category $\mathbf{C}^T$ is less clearly related to programming languages, but it is a very important category from at least three perspectives:

- General abstract nonsense: a given monad T can be decomposed as an adjunction $T = R \circ L$ in more than one way, where $L : \mathbf{C} \to \mathbf{D}$ and $R : \mathbf{D} \to \mathbf{C}$. It turns out that $\mathbf{C}^T$ is the "biggest" choice for $\mathbf{D}$: for any decomposition of T, there is a (unique) functor from $\mathbf{D}$ to $\mathbf{C}^T$ commuting with the adjunctions. (Likewise the Kleisli category turns out to be the *initial* choice, so there is a unique functor from $\mathbf{C}_T$ to $\mathbf{D}$.)

- Algebra and logic: we can think of a given monad T as being presented by some kind of algebraic theory (signature, operations, axioms), maybe generalized (e.g., with infinite arities). Then, each T-algebra $\alpha : TX \to X$ makes the object X into a model of an algebraic theory encoding T (note that a given algebraic theory can be described by different sets of operations and equations). Then, T-algebra morphisms are homomorphisms between models, so $\mathbf{C}^T$ can be viewed as the category of all models of the algebraic theory in $\mathbf{C}$.

- Programming languages: A T-algebra is a morphism $\alpha : TX \to X$. If we think of TX as representing the type of computations that return something of type X while possibly performing effects described by T, then the morphism α converts an effectful computation into a pure value. We can think of α as a *handler* that implements the effects that a computation in TX might perform.

We'll give a sense of some of these angles in the following exercises.

1. Suppose that $\mathbf{C}$ has products. Show that $\mathbf{C}^T$ also has products.

   It turns out that for any algebraic theory, the class of models is always closed under homomorphic images, subalgebras, and products. The converse is also true (this is also called Birkhoff's Variety Theorem, or Birkoff's HSP Theorem). We are considering a much more general notion of algebraic theory (and also models in a general category $\mathbf{C}$), but the class of models is still closed under products as long as $\mathbf{C}$ has products.

2. Let $W : \mathbf{Set} \to \mathbf{Set}$ be the endofunctor that takes a set X to the set of finite words on X, i.e.:

   $$WX \triangleq \{w_1 \dots w_n \mid w_i \in X\}$$

   Note that the word may be empty ($n = 0$). Spell out the definition of this monad (functor, natural transformations, etc.), and prove that $\mathbf{Set}^W$ is equivalent to the category of monoids $\mathbf{Mon}$ (in $\mathbf{Set}$). So although the monad W isn't obviously related to monoids, this monad turns out to be presented by the algebraic theory of monoids.

---

1. Assume $\mathbf{C}$ has products. Let $(A, \alpha : TA \to A)$ and $(B, \beta : TB \to B)$ be T-algebras in $\mathbf{C}^T$. The product object in $\mathbf{C}^T$ is $(A \times B, \gamma : T(A \times B) \to A \times B)$, where $\gamma = \langle \alpha \circ T\pi_A, \beta \circ T\pi_B \rangle$, and $\pi_A : A \times B \to A$, $\pi_B : A \times B \to B$ are the projections in $\mathbf{C}$. Verify that $(A \times B, \gamma)$ is a T-algebra:

   - Unit law: $\gamma \circ \eta_{A\times B} = id_{A\times B}$. Since $\alpha \circ \eta_A = id_A$ and $\beta \circ \eta_B = id_B$, and $\eta_{A\times B} = \langle \eta_A \circ \pi_A, \eta_B \circ \pi_B \rangle$ (naturality of η), then $\gamma \circ \eta_{A\times B} = \langle \alpha \circ T\pi_A \circ \eta_{A\times B}, \beta \circ T\pi_B \circ \eta_{A\times B} \rangle = \langle \alpha \circ \eta_A \circ \pi_A, \beta \circ \eta_B \circ \pi_B \rangle = \langle \pi_A, \pi_B \rangle = id_{A\times B}$

   - Multiplication law: $\gamma \circ \mu_{A\times B} = \gamma \circ T\gamma$. Left side: $\gamma \circ \mu_{A\times B} = \langle \alpha \circ T\pi_A \circ \mu_{A\times B}, \beta \circ T\pi_B \circ \mu_{A\times B} \rangle = \langle \alpha \circ \mu_A \circ T^2\pi_A, \beta \circ \mu_B \circ T^2\pi_B \rangle$ (naturality of μ). Right side: $\gamma \circ T\gamma = \langle \alpha \circ T\pi_A \circ T\gamma, \beta \circ T\pi_B \circ T\gamma \rangle = \langle \alpha \circ T(\alpha \circ T\pi_A), \beta \circ T(\beta \circ T\pi_B) \rangle = \langle \alpha \circ T\alpha \circ T^2\pi_A, \beta \circ T\beta \circ T^2\pi_B \rangle$. Since $\alpha \circ \mu_A = \alpha \circ T\alpha$ and similarly for β, they match.

   The projections $\pi_A : (A \times B, \gamma) \to (A, \alpha)$ and $\pi_B : (A \times B, \gamma) \to (B, \beta)$ are algebra morphisms because $\pi_A \circ \gamma = \alpha \circ T\pi_A$ and $\pi_B \circ \gamma = \beta \circ T\pi_B$ by construction. For universality: given algebra morphisms $f : (C, \kappa) \to (A, \alpha)$, $g : (C, \kappa) \to (B, \beta)$, the unique map in $\mathbf{C}$ is $h = \langle f, g \rangle : C \to A \times B$. Check h is an algebra morphism: $h \circ \kappa = \gamma \circ Th$. Since $f \circ \kappa = \alpha \circ Tf$, $g \circ \kappa = \beta \circ Tg$, then $\langle f, g \rangle \circ \kappa = \langle \alpha \circ Tf, \beta \circ Tg \rangle = \langle \alpha \circ T\pi_A \circ T\langle f, g \rangle, \beta \circ T\pi_B \circ T\langle f, g \rangle \rangle = \gamma \circ Th$. Thus, $\mathbf{C}^T$ has products.

2. The endofunctor $W : \mathbf{Set} \to \mathbf{Set}$ is $WX = \coprod_{n=0}^{\infty} X^n$, the set of finite lists over X, including the empty list []. For $f : X \to Y$, $Wf([x_1, \dots, x_n]) = [f(x_1), \dots, f(x_n)]$. The unit $\eta_X : X \to WX$, $\eta_X(x) = [x]$. The multiplication $\mu_X : W(WX) \to WX$, $\mu_X([l_1, \dots, l_k]) = l_1 + \dots + l_k$, where $+$ is concatenation. Verify monad laws:

- Left unit: $\mu \circ \eta W = id_W$, $\mu([l]) = l$.

- Right unit: $\mu \circ W\eta = id_W$, $\mu([[x_{11}], \ldots, [x_{kn}]]) = [x_{11}, \ldots, x_{kn}]$.

- Associativity: $\mu \circ \mu W = \mu \circ W\mu$, both flatten nested lists by repeated concatenation.

A W-algebra is $(X, h : WX \to X)$ satisfying $h \circ \eta =$ and $h \circ \mu = h \circ Wh$. Define $e = h([]) \in X$, and for $x, y \in X$, $x \cdot y = h([x, y])$. Then, unit laws: $x \cdot e = h([x, e]) = h([x] + []) = h \circ Wh([[x], []]) = h \circ \mu([[x], []]) = h([[x]]) = x$, similarly $e \cdot x = x$. Associativity: $x \cdot (y \cdot z) = h([x, h([y, z])]) = h \circ Wh([[x], [y, z]]) = h \circ \mu([[x], [y, z]]) = h([[x], [y, z]]) = h([x, y, z])$. Similarly, $(x \cdot y) \cdot z = h([x, y, z])$. Thus, $(X, \cdot, e)$ is a monoid. Conversely, for a monoid $(X, \cdot, e)$, define $h([x_1, \ldots, x_n]) = x_1 \cdot \cdots \cdot x_n$, with $h([]) = e$. This satisfies the algebra laws. Morphisms: algebra morphisms correspond to monoid homomorphisms. Thus, $\mathbf{Set}^W \simeq \mathbf{Mon}$.

Exercise 5 (Sparkly Star). Let $[\mathbf{C}, \mathbf{C}]$ be the category of *endofunctors* on $\mathbf{C}$, i.e., where objects are functors from $\mathbf{C}$ to itself, and morphisms are natural transformations. We can define a monoidal product via $F \otimes G \triangleq F \circ G$. (Note that this is definitely not a *symmetric* monoidal product, since the order of composition matters.)

1. Describe this monoidal structure on $[\mathbf{C}, \mathbf{C}]$. In particular, define the unit object in this category, and also describe the bifunctor structure (i.e., how $\otimes$ acts on morphisms in this category). You might want to review the optional exercises from A1 (although you don't need to complete them to do this problem).

2. Take a look at the definition of a monoid in a monoidal category. Show that a monad (T, μ, η) on $\mathbf{C}$ is the same thing as a monoid in the category of endofunctors $[\mathbf{C}, \mathbf{C}]$ with the monoidal product above.

While this may seem like a very convoluted way to describe a monad, it can be useful to generalize. For instance, if $(\mathbf{M}, \otimes, I)$ is a monoidal category, then a lax monoidal functor $T : \mathbf{M} \to [\mathbf{C}, \mathbf{C}]$ is the same thing as a $\mathbf{M}$-graded monad.

---

1. The category $[\mathbf{C}, \mathbf{C}]$ has endofunctors on $\mathbf{C}$ as objects and natural transformations as morphisms. The monoidal product $\otimes : [\mathbf{C}, \mathbf{C}] \times [\mathbf{C}, \mathbf{C}] \to [\mathbf{C}, \mathbf{C}]$ is defined on objects by $F \otimes G = F \circ G$. The unit object is the identity functor $I = \mathrm{Id}_{\mathbf{C}}$. On morphisms, for natural transformations $\alpha : F \Rightarrow F'$ and $\beta : G \Rightarrow G'$, define $\alpha \otimes \beta = \alpha \circ \beta : F \circ G \Rightarrow F' \circ G'$ as the horizontal composition $\alpha *_h \beta = F'\beta \circ \alpha G = \alpha G' \circ F\beta$. The associator $\alpha_{F,G,H} : (F \circ G) \circ H \Rightarrow F \circ (G \circ H)$ is the identity natural transformation, since functor composition is associative. Similarly, the left unitor $\lambda_F : \mathrm{Id} \circ F \Rightarrow F$ and right unitor $\rho_F : F \circ \mathrm{Id} \Rightarrow F$ are identities. This forms a monoidal category $([\mathbf{C}, \mathbf{C}], \circ, \mathrm{Id})$.

2. A monoid in the monoidal category $([\mathbf{C},\mathbf{C}], \circ, \mathrm{Id})$ consists of an object T (an endofunctor), a multiplication morphism $m : T \circ T \Rightarrow T$ (natural transformation $\mu : T^2 \Rightarrow T$), and a unit morphism $u : \mathrm{Id} \Rightarrow T$ (natural transformation $\eta : \mathrm{Id} \Rightarrow T$), such that the following:

   - Associativity: $m \circ (m \otimes \mathrm{id}_T) = m \circ (\mathrm{id}_T \otimes m)$, i.e., $\mu \circ (\mu T) = \mu \circ (T\mu)$.

   - Left unit: $m \circ (u \otimes \mathrm{id}_T) = \lambda_T$, but since λ is id, $\mu \circ (\eta T) = \mathrm{id}_T$.

   - Right unit: $m \circ (\mathrm{id}_T \otimes u) = \rho_T$, i.e., $\mu \circ (T\eta) = \mathrm{id}_T$.

   These are precisely the axioms for a monad (T, μ, η) on $\mathbf{C}$.

Exercise 6 (Sparkly Star). A strong monad comes equipped with natural transformations $t_{A,B} : A \otimes TB \to T(A \otimes B)$ and $t'_{A,B} : TA \otimes B \to T(A \otimes B)$. Using these maps, there are two ways to define a map $TA \otimes TB \to T(A \otimes B)$: applying t and then t', or vice versa. A strong monad is *commutative* if both maps are the same. Take a look at the definition of commutative monad on Wikipedia (or nLab).

Suppose we have judgments

$$\Gamma \vdash e_1 : TA \quad \text{and} \quad \Gamma \vdash e_2 : TB. \quad \text{and} \quad \Gamma, x : A, y : B \vdash e : TC$$

and suppose that T is a commutative monad. Show that the semantics of the following programs are equal:

$$\text{let } x \leftarrow e_1 \text{ in let } y \leftarrow e_2 \text{ in } e \quad \text{and} \quad \text{let } y \leftarrow e_2 \text{ in let } x \leftarrow e_1 \text{ in } e$$

Examples of commutative monads include powerset (non-determinism), maybe (partiality), and distribution (randomization). The writer and state monads are not commutative. [Hint: use naturality of $t_{A,B}$ and $t'_{A,B}$ at $[\![e_1]\!]$ and $[\![e_2]\!]$.]

I believe you understand this argument but it is not very easy to understand given what you've written

Since T is a commutative strong monad, the order of let-bindings for independent computations e_1 and e_2 does not affect the semantics of the program. The semantics of let $x \leftarrow e_1$ in let $y \leftarrow e_2$ in e is given by $\mu \circ T[\![e]\!] \circ Tt_{[\![\Gamma]\!] \times A, B} \circ T(\mathrm{id} \times ([\![e_2]\!] \circ \pi_1)) \circ t_{[\![\Gamma]\!], A} \circ (\mathrm{id} \times [\![e_1]\!])$. The semantics of the program with swapped bindings is similar, with the roles of A and B, and t and t', adjusted accordingly. Using the naturality of t and t' with respect to the morphisms $[\![e_1]\!]$ and $[\![e_2]\!]$, move the computations through the strengths in the compositions. The commutativity condition $\mu \circ Tt' \circ t = \mu \circ Tt \circ t'$ ensures that the sequencing of effects from e_1 and e_2 is independent of order. The nested application of strengths in one order can be rewritten using the braiding of the symmetric monoidal structure to express one bind in terms of the right strength t', allowing the commutativity condition to be applied to swap the order, and then rewriting back using naturality to match the other composition.
